

Whispered Speech LoRA • NGC Repack 工作指导

对象: 把 NGC 官方 `stt_en_fastconformer_hybrid_large_streaming_80ms.nemo` 适配进公司 `S_*` 体系, 跑通 LoRA 微调
编写日期: 2026-05-26 (执行日: 2026-05-27)
本地工作目录: `e:\phd\SE\code_R0I\CC_code\SS\code0526\`
远端工作目录: `/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/task3_0526/`
conda 环境: `nemo_env`

目录

1. 背景与今日进展回顾
2. 今日代码完整理解
3. 核心问题: NGC ckpt 与 `S_*` 体系的不兼容
4. `S_*` 私有方法 ↔ 上游 NeMo 一对一映射
5. 方案路径选择 (Path 1 直改 vs Path 2 Repack)
6. 完整 Repack 指导 (Phase 0 ~ Phase 6)
7. 今日实际遇到的环境问题: `numba-cuda` × `nvJitLink`
8. 明日工作 checklist
9. 参考资料

1. 背景与今日进展回顾

今天的目标是: 在 NGC 官方下载的 FastConformer Hybrid (RNN-T + CTC) 模型上, 注入 LoRA 做 whispered speech 域适应训练。代码框架沿用公司内部 `S_*` 体系的训练脚本。

整个流程触发了 **三层问题** (已逐层排查):

1. **第一层 • 类不匹配:** `restore_from` 后 `asr_model.joint` 是上游 `RNNTJoint`, 没有 `set_fast_rnnt` 方法 → `AttributeError`
2. **第二层 • 工程便利方法不存在:** `setup_aux_ctc` 上游也没有, 是公司类的扩展
3. **第三层 • 底层 JIT 环境:** 注释掉前两层的报错点后, 训练真正跑起来, 撞上 `numba-cuda` × `nvJitLink` 版本不兼容

明天要完成的核心工作: 把 NGC ckpt repack 成 `S_EncDecTransducerBPPEModel` 兼容格式 (永久解决第一、二层), 同时修复 `numba-cuda` 环境 (解决第三层)。

2. 今日代码完整理解

训练脚本: `code0526/whispered_speech_lora_finetune.py` (89 行)
配置文件: `code0526/configs/S_whispered_speech_lora_finetune.yaml` (108 行)

2.1 训练流程的五步

1. **模型加载与冻结:** 从 `cfg.asr_model_path` 加载 `.nemo`, `strict=False` 容忍权重 key 不对齐; 之后所有参数 `requires_grad=False` 全冻结
2. **LoRA 注入:** `r=8`, `lora_alpha=32`, `dropout=0.05`, 注入 6 个 Linear 层名 (`linear_q/k/v/out` + FFN 的 `linear1/linear2`), 只包装 `asr_model.encoder`
3. **训练准备:** `set_trainer`、关闭 `fast_rnnt`、挂 `aux_ctc`、`setup_training_data` / `setup_validation_data`
4. **训练:** `trainer.fit(asr_model)`
5. **导出:** `merge_and_unload()` 把 LoRA 增量合回原 Linear, `save_to` 导出标准 `.nemo`

2.2 YAML 关键设定

项	值	含义
compute_eval_loss	false	验证只算 WER 不算 transducer loss, 避免长样本爆显存
train_ds.batch_size / bucket_batch_size	8 / [6,2,2]	bucketing: ≤8s bs=6, ≤15s bs=2, ≤30s bs=2
aux_ctc.ctc_loss_weight	0.0	CTC 辅助损失完全关闭, 只用 RNN-T loss
val_check_interval	1600	每 1600 步验证一次
accumulate_grad_batches	16	有效 batch = bucket_bs × 16 × n_gpus
gradient_clip_val	0.0001	建议核查: FastConformer 一般用 1.0 或 0.5, 0.0001 几乎压平所有梯度
precision	32	全精度, LoRA 参数少不差显存

3. 核心问题: NGC ckpt 与 S_* 体系的不兼容

错误原文:

```
AttributeError: 'RNNTJoint' object has no attribute 'set_fast_rnnnt'
```

关键证据是 'RNNTJoint' 而不是 'S_RNNTJoint'。

3.1 为什么

NeMo 的 .nemo tarball 里有 model_config.yaml, 里面用 _target_ 字段指明每个子模块的 Python 类:

加载来源	joint 反序列化成什么	有 set_fast_rnnnt 吗
之前同事训出来的 .nemo	S_RNNTJoint (公司 fork)	✓
NGC 官方 hybrid 模型	上游 nemo.collections.asr.modules.RNNTJoint	✗

关键陷阱: restore_from(..., strict=False) 只是宽容处理权重 key 的不一致, 它不会把类换成 S_* 版本。官方 .nemo 里写的是哪个类, PyTorch 就把 joint 实例化成哪个类。

3.2 下游还会撞到的雷

```
asr_model.fast_rnnnt = False # 不报错 (Python 动态属性), 但也没用
asr_model.joint.set_fast_rnnnt(fast_rnnnt=False) # ← 第一个雷
asr_model.setup_aux_ctc(cfg.model) # ← 第二个雷: S_* 自定义方法
```

YAML 里这段也只在 S_fork 里存在:

```
aux_ctc:
  decoder:
    _target_: nemo.collections.asr.modules.S_taed_decoder.S_SimpleCTCDecoder
```

4. S_* 私有方法 ↔ 上游 NeMo 一对一映射

基于 NeMo 主仓源码 (带 file:line) 的实证对照:

公司库的写法	上游 NeMo 对应物	关系
asr_model.set_trainer(trainer)	ModelPT.set_trainer	同名
asr_model.fast_rnnnt = False	joint.fuse_loss_wer (属性)	改名
asr_model.joint.set_fast_rnnnt(False)	joint.set_fuse_loss_wer(False)	改名
asr_model.setup_aux_ctc(cfg.model)	无对应方法. 改用 hybrid 类, __init__ 里自动建	机制不同
asr_model.setup_training_data(...)	ModelPT.setup_training_data	同名
asr_model.setup_validation_data(...)	ModelPT.setup_validation_data	同名

4.1 set_fast_rnnnt ≡ set_fuse_loss_wer

证据： `RNNTJoint.set_fuse_loss_wer(fuse_loss_wer, loss=None, metric=None)`，位于 `nemo/collections/asr/modules/rnnt.py` 约 L2550。

作用： 把 "joint forward + RNN-T loss + WER 计算" 三步融合，按子批次 (`fused_batch_size`) 流式处理，**用速度换显存**。RNN-T 训练必备技巧——joint tensor 形状 `[B, T, U, V]`，词表 $V \approx 1024$ 时显存爆炸。

为什么是同一个开关： 你们仓库管它叫 `fast_rnnt` ("快/省内存版 RNN-T")，上游叫 `fuse_loss_wer` ("融合 loss 和 WER")。同一个开关。上游 `EncDecRNNTBPEModel.__init__` (`rnnt_bpe_models.py` L142-145) 自动判断：若 `joint.fuse_loss_wer == True`，就调 `joint.set_loss(self.loss)` 和 `joint.set_wer(self.wer)`。

4.2 `setup_aux_ctc` —— 上游没有这个方法

证据： 上游 `EncDecHybridRNNTCTCModel` (`hybrid_rnnt_ctc_models.py`) 在 `__init__` 里直接：

- 从 `cfg.aux_ctc.decoder` 实例化 `self.ctc_decoder`
- 构造 `self.ctc_loss = CTCLoss(...)`，按 `cfg.aux_ctc.ctc_reduction` 配置
- 读 `self.ctc_loss_weight = cfg.aux_ctc.get("ctc_loss_weight", 0.5)`
- 在 `training_step` 里按 `loss = (1-w)*rnnt_loss + w*ctc_loss` 混合

关键差异： 上游 aux CTC 是 hybrid 模型类的一等公民，构造期就建好；公司库则单独抽出来一个 `setup_aux_ctc(cfg)` 方法，可以"事后给纯 transducer 模型挂上 CTC 头"。这是工程便利，不是性能优化。

5. 方案路径选择

Path 1 • 用上游 hybrid 类，改训练脚本

```
from nemo.collections.asr.models import EncDecHybridRNNTCTCBPEModel

asr_model = EncDecHybridRNNTCTCBPEModel.restore_from(
    asr_model_path, map_location='cpu', strict=False,
)
# ... LoRA 注入逻辑不变 ...
asr_model.set_trainer(trainer)
if hasattr(asr_model.joint, "set_fuse_loss_wer"):
    asr_model.joint.set_fuse_loss_wer(fuse_loss_wer=False)
# setup_aux_ctc 那行删 — hybrid 类 __init__ 已经建好 ctc_decoder/ctc_loss
asr_model.setup_training_data(train_data_config=cfg.model.train_ds)
asr_model.setup_validation_data(val_data_config=cfg.model.validation_ds)
```

优点：改动最小，不需要 repack。

缺点：下游所有 inference / eval / 蒸馏脚本（建立在 S_* 体系上的）都得跟着改一遍。

Path 2 • Repack NGC ckpt 进 S_* 体系（推荐）

优点：一次性投入，训练脚本零修改，下游所有脚本继续走 S_* 管线。

缺点：需要写一个 repack 脚本 + 严格验证。

决策：走 Path 2。明天的工作重点是把 Phase 1 ~ Phase 5 跑通。

6. 完整 Repack 指导

Phase 0 • 决策与前置 (5 分钟)

开工前确认四件事：

1. `S_RNNTJoint` 完整 `import` 路径 (你已 `grep` 到, 明天填进脚本)
2. `S_EncDecTransducerBPEModel.__init__(cfg)` 的强制字段：

```
grep -n "self\." /path/to/S_transformer_transducer_bpe_models.py | head -60
```

重点看 `__init__` 里有没有强读 `cfg.aux_ctc`、`cfg.ctc_decoder`。

3. `S_RNNTJoint.__init__` 跟上游 `RNNTJoint` 的签名差异：

```
grep -n "def __init__" /path/to/S_rnnt.py
```

4. NGC 那个 `.nemo` 里 `cfg.encoder._target_` —— 后面要决定是否一起换

Phase 1 • Inspection (摸清两端)

为什么必须先摸清：不能直接抄 NGC `cfg` 然后修修补补。NGC 是 hybrid, 多 `ctc_decoder` 块、可能有 `decoder_losses_weights`；流式参数 (`att_context_size`, `conv_context_size`) 藏在 `encoder` 深处, 漏一个推理时就不是流式了。

放到 `code0526/scripts/01_inspect_src.py`：

```
"""Phase 1: 把 NGC hybrid ckpt 完整 dump 出来。只读, 零副作用。"""
import json
from pathlib import Path
from omegaconf import OmegaConf
from nemo.collections.asr.models import EncDecHybridRNNTCTCBPEModel

SRC = "/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/pretrained_model/stt_en_fastconformer_hybrid_large_streaming_80ms.nemo"
OUT = Path("./ngc_dump"); OUT.mkdir(exist_ok=True)

src = EncDecHybridRNNTCTCBPEModel.restore_from(SRC, map_location="cpu")

# (a) 完整 cfg
(OUT / "src_cfg.yaml").write_text(OmegaConf.to_yaml(src.cfg))

# (b) state_dict key/shape 按前缀分组
sd = src.state_dict()
groups = {}
for k, v in sd.items():
    groups.setdefault(k.split(".")[0], []).append((k, tuple(v.shape)))
with open(OUT / "src_sd.txt", "w") as f:
    for prefix in sorted(groups):
        f.write(f"\n### {prefix} ({len(groups[prefix])} tensors)\n")
        for k, s in groups[prefix]:
            f.write(f"    {k:80s} {s}\n")

# (c) 关键尺寸
info = {
    "vocab_size": src.tokenizer.vocab_size,
    "encoder_target": src.cfg.encoder._target_,
    "encoder_d_model": src.cfg.encoder.d_model,
    "encoder_n_layers": src.cfg.encoder.n_layers,
    "encoder_n_heads": src.cfg.encoder.n_heads,
    "encoder_subsampling_factor": src.cfg.encoder.subsampling_factor,
    "encoder_att_context_size": src.cfg.encoder.get("att_context_size", None),
    "encoder_conv_context_size": src.cfg.encoder.get("conv_context_size", None),
    "decoder_target": src.cfg.decoder._target_,
    "decoder_pred_hidden": src.cfg.decoder.prednet.pred_hidden,
    "joint_target": src.cfg.joint._target_,
    "joint_hidden": src.cfg.joint.jointnet.joint_hidden,
    "joint_num_classes": src.cfg.joint.num_classes,
}
(OUT / "key_dims.json").write_text(json.dumps(info, indent=2))
print(json.dumps(info, indent=2))
```

Pass 标准: `key_dims.json` 里的真实值跟你训练 yaml 的 `model_defaults` 对得上。如果对不上, 以 **NGC 的真实值为准**, 把 yaml 同步过去。

Phase 2 • 设计 dst_cfg

原则：copy → modify, 不要 from scratch。NeMo 子模块靠 `hydra.utils.instantiate` , 漏一个字段会用默认值, 导致"实例化不报错但推理静默错误"。

改动	改成	理由
顶层 <code>_target_</code>	<code>S_EncDecTransducerBPEModel</code> 全路径	repack 核心目的
<code>joint._target_</code>	你 grep 到的 <code>S_RNNTJoint</code> 路径	让 <code>set_fast_rnn</code> 不再 <code>AttributeError</code>
删 <code>ctc_decoder</code> 块	—	hybrid 才有, 纯 transducer 没有
删 <code>decoder_losses_weights</code>	—	hybrid 用的混合 loss 权重
删 <code>aux_ctc</code> (如果 src 有)	—	让训练 yamll 提供 (你已经有)
<code>encoder</code> 块	原样不动	流式参数全在这里
<code>decoder</code> 块 (predictor)	原样不动	权重维度必须匹配
<code>tokenizer</code> 块	原样 (暂时)	<code>save_to</code> 会重新打包
<code>decoding</code> 块	原样	推理时用
<code>train_ds / validation_ds / test_ds / optim</code>	可以删	训练时 Hydra 会重新提供

关于 `decoder` 子模块要不要换 `_target_`: 默认不换。grep 训练脚本里有没有调 `asr_model.decoder.set_xxx(...)` , 没有就保持上游。

关于 `aux_ctc` 留还是删: 依赖 `S_EncDecTransducerBPEModel.__init__` 。如果 `__init__` 不强读 `cfg.aux_ctc` → 完全删掉; 如果强读 → 留一个 dummy 块 (`ctc_loss_weight: 0.0` , `ctc_decoder` 用 `S_SimpleCTCDecoder`) 。

Phase 3 • Repack 实施

放到 `code0526/scripts/02_repack.py` :

```

"""Phase 3: NGC hybrid → S_EncDecTransducerBPEModel."""
import copy
import torch
from omegaconf import OmegaConf, open_dict
from nemo.collections.asr.models import EncDecHybridRNNTCTCBPEModel
from nemo.collections.asr.models.S_transformer_transducer_bpe_models import (
    S_EncDecTransducerBPEModel,
)

SRC = "/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/pretrained_model/stt_en_fastconformer_hybrid_large_streaming_80ms.ne
DST = "/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/pretrained_model/stt_en_fastconformer_S_repacked.nemo"

# ← 改成你 grep 到的真实路径
S_JOINT_TARGET = "nemo.collections.asr.modules.S_rnnnt.S_RNNTJoint"
S_MODEL_TARGET = ("nemo.collections.asr.models."
                  "S_transformer_transducer_bpe_models.S_EncDecTransducerBPEModel")

# [1] 装载源模型
# 为什么用 hybrid 类装：只有它的 __init__ 能正确解析 NGC ckpt 里的
# ctc_decoder / aux_ctc 块。换成其他类会丢权重或构造失败。
print("[1/6] Loading source as EncDecHybridRNNTCTCBPEModel ...")
src = EncDecHybridRNNTCTCBPEModel.restore_from(SRC, map_location="cpu")
src.eval()

# [2] 构造 dst_cfg
# 为什么 deepcopy: 避免 src 后续被 dst 共享引用污染。
print("[2/6] Building dst_cfg ...")
dst_cfg = copy.deepcopy(src.cfg)

with open_dict(dst_cfg):
    dst_cfg._target_ = S_MODEL_TARGET
    dst_cfg.joint._target_ = S_JOINT_TARGET

    for k in ("ctc_decoder", "decoder_losses_weights"):
        if k in dst_cfg:
            del dst_cfg[k]

    if "aux_ctc" in dst_cfg:
        del dst_cfg["aux_ctc"]
        # 如果构造期报缺 aux_ctc, uncomment 下面:
        # dst_cfg.aux_ctc = OmegaConf.create({
        #     "ctc_loss_weight": 0.0, "use_cer": False, "ctc_reduction": "mean",
        #     "decoder": {
        #         "_target_": "nemo.collections.asr.modules.S_taed_decoder.S_SimpleCTCDecoder",
        #         "feat_in": None, "num_classes": -1,
        #     },
        # })

    for k in ("train_ds", "validation_ds", "test_ds", "optim"):
        if k in dst_cfg:
            del dst_cfg[k]

OmegaConf.save(dst_cfg, "/tmp/dst_cfg.yaml")
print("    /tmp/dst_cfg.yaml written -- diff against ngc_dump/src_cfg.yaml")

# [3] 实例化 dst
# 为什么这一步要在 src 还活着的时候做:
# src.cfg.tokenizer.dir 指向 src restore 时的 tmp 解压路径。
# 如果 src 被 GC, tmp 目录被清理, dst 实例化时 tokenizer 加载会失败。
print("[3/6] Instantiating S_EncDecTransducerBPEModel(cfg=dst_cfg) ...")
dst = S_EncDecTransducerBPEModel(cfg=dst_cfg)
dst.eval()

# [4] 拷贝权重
# 为什么 strict=False:
# - 期望 unexpected: ctc_decoder.* (hybrid 多出来的 CTC 头)
# - 期望 missing: 0
print("[4/6] Loading state_dict ...")
missing, unexpected = dst.load_state_dict(src.state_dict(), strict=False)

print(f"\n--- missing ({len(missing)}) ---")
for k in missing: print(f" {k}")
print(f"\n--- unexpected ({len(unexpected)}) ---")
for k in unexpected[:30]: print(f" {k}")
if len(unexpected) > 30: print(f" ... +{len(unexpected)-30} more")

# 硬断言 — 不通过就停
non_ctc_unexpected = [k for k in unexpected if not k.startswith("ctc_decoder.")]

```



```

assert len(non_ctc_unexpected) == 0, (
    f"Got unexpected keys outside ctc_decoder: {non_ctc_unexpected}\n"
    f"This means dst has a different module layout than src. STOP and inspect."
)
assert len(missing) == 0, (
    f"Missing keys: {missing}\n"
    f"S_RNNTJoint may have extra layers, or _target_ swap was wrong."
)

# [5] 前向数值等价
# 为什么这一步必须做: load_state_dict 成功只保证"权重装进去了",
# 不保证"前向逻辑等价"。这是金标准。
print("[5/6] Forward parity check ...")
torch.manual_seed(0)
wav = torch.randn(1, 16000 * 4)
wav_len = torch.tensor([wav.shape[1]])

with torch.no_grad():
    f_s, fl_s = src.preprocessor(input_signal=wav, length=wav_len)
    f_d, fl_d = dst.preprocessor(input_signal=wav, length=wav_len)
    assert torch.allclose(f_s, f_d), "preprocessor mismatch"

    e_s, _ = src.encoder(audio_signal=f_s, length=fl_s)
    e_d, _ = dst.encoder(audio_signal=f_d, length=fl_d)
    err_enc = (e_s - e_d).abs().max().item()

    y = torch.zeros(1, 1, dtype=torch.long)
    d_s, _, _ = src.decoder(targets=y, target_length=torch.tensor([1]))
    d_d, _, _ = dst.decoder(targets=y, target_length=torch.tensor([1]))
    err_dec = (d_s - d_d).abs().max().item()

    j_s = src.joint(encoder_outputs=e_s, decoder_outputs=d_s)
    j_d = dst.joint(encoder_outputs=e_d, decoder_outputs=d_d)
    err_jnt = (j_s - j_d).abs().max().item()

print(f"    encoder  max|delta| = {err_enc:.2e}")
print(f"    decoder  max|delta| = {err_dec:.2e}")
print(f"    joint    max|delta| = {err_jnt:.2e}")

TOL = 1e-5
assert err_enc < TOL, f"encoder parity broken (delta={err_enc:.2e})"
assert err_dec < TOL, f"decoder parity broken (delta={err_dec:.2e})"
assert err_jnt < TOL, f"joint parity broken (delta={err_jnt:.2e})"

# [6] 保存
print("[6/6] Saving repacked .nemo ...")
dst.save_to(DST)
print(f"\nDone: {DST}")

```

Pass 标准: 所有 assert 通过。三个 `max|delta|` 都 < 1e-5。任何一条不通过 → 去 Phase 6 看 debug 路径。

Phase 4 • 端到端 sanity

为什么前向数值对了还要再测：Phase 3 只验证了纯 PyTorch forward。NeMo 的 transcribe() 走另一条路径（包括 decoding strategy、beam search、batch padding）。

放到 code0526/scripts/03_sanity.py：

```
"""Phase 4: 端到端推理 sanity."""
from nemo.collections.asr.models import EncDecHybridRNNTCTCBPEModel
from nemo.collections.asr.models.S_transformer_transducer_bpe_models import (
    S_EncDecTransducerBPEModel,
)

SRC = "/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/pretrained_model/stt_en_fastconformer_hybrid_large_streaming_80ms.nemo"
DST = "/home/CORP/d.zhang/d-zhang/task_train_ASR_adapter/pretrained_model/stt_en_fastconformer_S_repacked.nemo"

# 找 3 条 chains_whisper_zdz.json 里的真实音频
TEST_WAVS = [
    "/some/path/sample1.wav",
    "/some/path/sample2.wav",
    "/some/path/sample3.wav",
]

src = EncDecHybridRNNTCTCBPEModel.restore_from(SRC, map_location="cuda").eval()
dst = S_EncDecTransducerBPEModel.restore_from(DST, map_location="cuda").eval()

src_out = src.transcribe(TEST_WAVS, batch_size=1)
dst_out = dst.transcribe(TEST_WAVS, batch_size=1)

def _txt(x):
    return x.text if hasattr(x, "text") else x

for i, (a, b) in enumerate(zip(src_out, dst_out)):
    a, b = _txt(a), _txt(b)
    mark = "OK" if a == b else "FAIL"
    print(f"[{mark}] [{i}] src: {a}")
    print(f"          [{i}] dst: {b}")
```

Pass 标准：三句转写逐字一致。不一致但 Phase 3 通过 → 检查 dst_cfg.decoding 是不是抄全了。

Phase 5 • 接入训练流水线

只动一处 —— configs/S_whispered_speech_lora_finetune.yaml#L2：

```
asr_model_path: "/ ... /pretrained_model/stt_en_fastconformer_S_repacked.nemo"
```

训练脚本零修改。这就是 repack 路线的全部价值。

冒烟测试跑 50 step，关心三件事：

现象	期望	不期望时怎么办
set_fast_rnnt 那行是否 crash	不 crash	S_RNNTJoint 没装上 → 回 Phase 2 检查 _target_
Trainable: X.XXM / Total: Y.YYM	Total ≈ 116.7M	变了说明子模块结构变了
前 20 step 的 train loss	从合理初始值缓慢下降	loss=inf/nan → grad_clip 0.0001 太小，调到 1.0

Phase 6 • 常见失败模式

现象	根因	修法
Phase 3 missing 里有 joint.*	S_RNNTJoint.__init__ 多了层 / hidden 不同	读源码补字段，或换上游
Phase 3 missing 里有 encoder.layers.*	dst_cfg.encoder 没原样抄	回 Phase 2 deepcopy
Phase 3 unexpected 有非 ctc_decoder.*	hybrid src 还有其他 head	看 src_sd.txt 挨个判断
err_enc ≈ 1e-2	encoder 流式参数丢了	diff dst/src cfg.encoder 逐字段
err_jnt ≈ 1e-2	S_RNNTJoint.forward 改了逻辑	修 S_RNNTJoint 跟上游对齐
Phase 4 转写不一致	decoding 配置漂移	检查 dst.cfg.decoding
实例化报 tokenizer 不存在	src 已被 GC, tmp 目录清理	Phase 3 整脚本一次跑，别分两次
构造期报缺 aux_ctc	__init__ 强读了 aux_ctc	Phase 2 uncomment dummy 块

全程核心理念：Repack 不是"改 cfg 然后期望它工作"，而是**把源模型当 ground truth，逐层验证 dst 跟它数值等价**。Phase 3 那个 1e-5 阈值是契约 —— 过了下游放心，不过则**任何 loss 异常都很可能是 repack 偷偷出错的回响**。

7. 今日实际遇到的环境问题

7.1 错误现场

当三行 `S_*` 私有调用被注释后，训练真正跑起来（Epoch 0: 0% | | 0/2473），随即崩在：

```
cuda.bindings.nvjitlink.nvJitLinkError: ERROR_INTERNAL (6)
Linker error log: ERROR 4 in nvvmAddNVVMContainerToProgram,
                  may need newer version of nvJitLink library
```

7.2 诊断

这是 NeMo 已知 bug：GitHub Issue NVIDIA-NeMo/NeMo#15155 — `numba-cuda > 0.15.1` 跟 NeMo RNN-T 训练不兼容。原因链：

```
trainer.fit(asr_model)
  → forward → joint(...) → RNNTLoss
  → warprnnt_numba 后端 JIT 编译 CUDA kernel
  → 调 nvJitLink 装载 PTX
  → numba-cuda > 0.15.1 用了较新的 NVVM 容器格式
  → 已安装的 nvJitLink 不认这个容器格式
  → ERROR 4 / ERROR_INTERNAL(6)
```

跟 LoRA 代码、跟 `S_*` 类、跟 NGC 模型完全无关。是纯环境问题。注释那三行的修改是正确的，保留。

7.3 修复（按成本递增）

修复 1 • 降级 numba-cuda（首选）

```
# 确认环境
which python      # 应该看到 .../nemo_env/bin/python

# 看当前版本
pip show numba-cuda | grep Version
pip show nvidia-nvjitlink-cu12 | grep Version

# 降级（--no-deps 避免动 numba 主包）
pip install --no-deps "numba-cuda==0.15.1"
```

修复 2 • 升级 nvJitLink（修复 1 不行再试）

```
pip install --upgrade nvidia-nvjitlink-cu12

# 但要先看 driver 版本撑得住吗
nvidia-smi | grep "CUDA Version"
# 12.4+ → nvjitlink 12.4+ 没问题
# 12.0-12.3 → nvjitlink 不要超过对应版本
```

修复 3 • 禁用 numba 新绑定（兜底）

```
# 必须在 import numba 之前设
NUMBA_CUDA_USE_NVIDIA_BINDING=0 CUDA_VISIBLE_DEVICES=0 python whispered_speech_lora_finetune.py
```

修复 4 • 换 RNN-T loss 后端（最后手段）

在 yaml 里 `model`：加：

```
model:
  loss:
    loss_name: "warprnnt"
```

要求装了 `warp-transducer`。可能 `nemo_env` 里没有，要先 `pip install warprnnt-pytorch`。

建议执行顺序：先 1 → 不行加 3 → 还不行撤销 1 做 2 → 最后才是 4。每次只改一个变量，方便回滚。

8. 明日工作 checklist

上午: 环境修复 + Phase 1 inspection

- [] SSH 到远端, 确认在 `nemo_env` conda 环境
- [] `pip show numba-cuda` 看版本, 记下来
- [] `pip install --no-deps "numba-cuda==0.15.1"`
- [] 跑一次原始训练脚本 (带那三行注释), 看 numba 错误是否消失
- [] 若仍报错, 按 7.3 修复 2/3/4 逐个尝试
- [] 环境通了之后, 把 `code0526/scripts/01_inspect_src.py` 拷贝到 `task3_0526`, 跑出 `ngc_dump/`
- [] 把 `key_dims.json` 跟 yaml 的 `model_defaults` 对比, 记下差异

下午: Phase 2-3 设计与 Repack

- [] `grep` 出 `S_RNNTJoint` 真实路径 (已知, 填进脚本)
- [] `cat S_EncDecTransducerBPEModel.__init__` 源码, 判断 `aux_ctc` 是不是强制字段
- [] 编写 `code0526/scripts/02_repack.py`, 按 Phase 3 模板
- [] 跑 `repack`, 确保三个 `assert` 全过
- [] 把生成的 `stt_en_fastconformer_S_repacked.nemo` 路径记下

傍晚: Phase 4-5 验证 + 冒烟训练

- [] 跑 `03_sanity.py`, 确认 `src/dst transcribe` 一致
- [] 修改 yaml 第 2 行的 `asr_model_path` 指向 `repacked.nemo`
- [] 跑训练 50 step, 看 `loss` 走势
- [] 若 `loss=nan`, 把 `gradient_clip_val` 从 0.0001 调到 1.0 重试
- [] 跑通后保存当前 yaml / 脚本到 `code0526/`, 本地 PyCharm 同步

结束前:

- [] 把今天的进展写进 `code0526/docs/progress_20260527.md`
- [] 把 `repack` 后的 `.nemo` 备份到一个安全位置 (这文件是后续所有实验的起点)

9. 参考资料

- **NeMo Issue #15155** — `speech_to_text_hybrid_rnnt_ctc_bpe.py` not working with `numba-cuda>0.15.1`
<https://github.com/NVIDIA-NeMo/NeMo/issues/15155>
- **NeMo Issue #15331** — NeMo 2.6/2.7 incompatible with `numba-cuda`
<https://github.com/NVIDIA-NeMo/NeMo/issues/15331>
- **numba Issue #10353** — 同样的 `nvJitLink` 错误在非 NeMo 上下文
<https://github.com/numba/numba/issues/10353>
- **numba-cuda Installation** (NUMBA_CUDA_USE_NVIDIA_BINDING 文档)
<https://nvidia.github.io/numba-cuda/user/installation.html>
- **RNNTJoint 源码** (`set_fuse_loss_wer` 在约 L2550)
<https://github.com/NVIDIA-NeMo/NeMo/blob/main/nemo/collections/asr/modules/rnnt.py>
- **EncDecRNNTBPEModel** (`auto-wire fuse_loss_wer` 在 L142-145)
https://github.com/NVIDIA-NeMo/NeMo/blob/main/nemo/collections/asr/models/rnnt_bpe_models.py
- **EncDecHybridRNNTCTCModel** (`aux_ctc` 在 `__init__`)
https://github.com/NVIDIA-NeMo/NeMo/blob/main/nemo/collections/asr/models/hybrid_rnnt_ctc_models.py
- **FastConformer Hybrid 训练 yaml 参考**
https://github.com/NVIDIA-NeMo/NeMo/blob/main/examples/asr/conf/fastconformer/hybrid_transducer_ctc/fastconformer_hybrid_transducer_ctc_bpe.yaml
- **NeMo Discussion #6295** — `fuse_loss_wer` semantics
<https://github.com/NVIDIA-NeMo/NeMo/discussions/6295>